

A Deep Learning-Based Task Offloading Method for Mechanical Vehicles in Edge Networks

Guangbo Xiang ^{*,†,‡} Huapo Jia,^{*,†,§} Peifang Xin,^{*,¶} Lifang Fan^{*,||}
and Xuefeng Wang^{*,**}

^{*}*College of Mechanical Engineering,
Zhengzhou University of Science and Technology,
Zhengzhou 450064, P. R. China*

[†]*Zhengzhou Industrial Design Center of Intelligent Equipment,
Zhengzhou 450064, P. R. China*

[‡]*guangboxiang_22@163.com*

[§]*jiahuapo509@163.com*

[¶]*haifang0115@163.com*

^{||}*fanlifang89@163.com*

^{**}*wangxuefeng202204@163.com*

Received 25 December 2024

Accepted 14 June 2025

Published 4 September 2025

With the rapid development of vehicular networks, the amount of data generated by vehicles has increased sharply. However, how to efficiently and intelligently realize task offloading for mechanical vehicles in vehicular networks is still a hot and difficult topic in current research. This study first analyzes the dynamic nature of the network environment, the limitation of computing resources and the diversity of task types. Then, it proposes a deep learning model based on variational autoencoders-long short-term memory (VAE-LSTM), to predict and decide on task offloading strategies in networked vehicles. The VAE-LSTM model combines the generation capability of VAE with the time series processing capability of LSTM. Such a design can accurately capture the changing trend of network state and task requirements. To verify the performance of the model, this work makes comparison experiments by comparing the VAE-LSTM model with other deep learning models. The experimental results show that the VAE-LSTM model is superior to other models in terms of prediction accuracy and stability, and can significantly reduce the mean absolute error of task offloading, and improve the resource utilization and task execution efficiency of the edge computing system. It meets the real-time and reliability requirements of the vehicular network system.

Keywords: Vehicle engineering; transportation engineering; mechanical vehicles; deep learning; task offloading.

[‡]Corresponding author.

1. Introduction

With the rapid development of the Internet of Things, big data and artificial intelligence technologies, vehicle ad hoc networks (VANETs), as an important emerging technology, are profoundly changing the face of transportation and intelligent transportation systems.¹ The Internet of Vehicles connects vehicles, roadside infrastructure and cloud platforms to achieve real-time communication between vehicles (V2V), communication between vehicles and infrastructure (V2I) and communication between vehicles and pedestrians (V2P), providing powerful technical support for applications such as autonomous driving, intelligent traffic management, remote vehicle monitoring and multimedia services.^{2,3} The rapid development of the Internet of Vehicles has also brought many challenges, among which the most prominent is the efficient allocation of computing resources. Modern vehicles are equipped with a large number of sensors and complex computing devices, which can generate massive amounts of data in real-time, such as high-definition camera images, LiDAR data and vehicle driving status information.⁴ These data need to be processed and analyzed in a short period of time to support functions, such as autonomous driving decision-making, traffic flow optimization and vehicle fault diagnosis. However, the local computing resources of vehicles are limited, especially when facing complex computing tasks, which may lead to increased processing latency, high energy consumption and decreased system performance.⁵

In order to meet this challenge, edge computing, as a new computing paradigm, has been introduced into the Internet of Vehicles environment. Edge computing can significantly reduce data transmission delay and improve the real-time performance and responsiveness of the system by sinking computing resources from the cloud to edge nodes near the data source.⁶ However, the resources of edge computing nodes are also limited, which cannot meet the processing requirements of all tasks. Therefore, how to allocate computing tasks reasonably among vehicles, ES and cloud servers has become an urgent problem to be solved.

In recent years, task offloading technology has become a key means of solving the problem of computing resource allocation in the Internet of Vehicles. The basic idea of task offloading is to offload some computing tasks on the vehicle to ES or cloud servers for execution, thereby fully utilizing external computing resources and reducing the local computing burden on the vehicle.⁷ However, task offloading faces many challenges, including how to choose appropriate offloading targets, how to dynamically adjust offloading strategies under different network conditions and resource states, and how to balance multi-objective optimization problems such as task completion time, energy consumption, and cost.⁸ Deep learning technology has been introduced into the research of task offloading for connected vehicles. Deep learning models, such as neural networks, reinforcement learning and generative adversarial networks, can automatically learn the complex relationships between task features and network states, providing intelligent support for task offloading decisions. For example, through reinforcement learning algorithms, vehicles can

dynamically adjust task offloading strategies based on real-time network and resource status to minimize task completion time; through deep neural networks, vehicles can model the computational requirements, data volume and priority of tasks, providing a more accurate basis for task offloading decisions.⁹

Although some progress has been made in the field of task offloading for connected vehicles, there are still many urgent issues that need to be addressed. First, most existing task offloading methods only focus on optimizing task completion time, while neglecting other key indicators such as energy consumption and cost. In practical applications, energy consumption and cost are important considerations for the operation of vehicle networking systems.¹⁰ How to achieve multi-objective optimization between task completion time, energy consumption and cost remains an urgent problem to be solved. Second, the connected car environment is highly dynamic, with real-time changes in the vehicle's driving path, network topology and resource status of ES. Most existing task offloading methods assume that task offloading decisions are one-time and cannot adapt to such dynamic changes.^{11,12} How to design a dynamic adaptive task offloading mechanism that enables vehicles to dynamically adjust task offloading strategies based on real-time network status and task requirements is an important direction of current research. Finally, task offloading in the Internet of Vehicles involves sensitive data of vehicles, such as driving trajectories and driving habits. Privacy protection is an important issue in practical applications.¹³ How to protect the privacy of vehicle data during task offloading without affecting system performance is a current research challenge.

The offloading methods of innovative tasks for the Internet of vehicles in this study are as follows:

- (1) The study proposes a task offloading strategy based on multimodal data fusion, which integrates real-time vehicle driving data, network status data and edge server load data to construct a comprehensive feature vector.
- (2) The study introduces a dynamic adaptive task offloading mechanism that utilizes reinforcement learning algorithms to enable vehicles to dynamically adjust task offloading strategies based on real-time network status and task requirements. After each task is unloaded, the vehicle updates the parameters of the deep learning model based on feedback information, thereby achieving rapid adaptation to dynamic environments.

2. Literature Review

With the rapid development of information technology, car networking, as the core component of an intelligent transportation system, is gradually becoming the focus of academic and industrial attention. Through the information interaction between vehicles and vehicles, vehicles and infrastructure and vehicles and people, vehicle networking realizes an intelligent and efficient traffic system. However, with the wide application of vehicle networking technology, the amount of data generated by

vehicles has increased dramatically, and the demand for computing resources is also growing. Although the traditional cloud computing model has powerful data processing capabilities, it is difficult to meet the high requirements of real-time and reliability of vehicle networking due to network delay and bandwidth bottleneck. Therefore, edge computing, as a distributed computing architecture, migrates computing tasks and data storage from the cloud to devices at the edge of the network, realizes localized data processing, reduces network delay, improves real-time and security of data processing, and gradually becomes an important means to solve the bottleneck of computing resources in the Internet of vehicles.

Liu *et al.*¹⁴ proposed a joint offloading strategy for PC-5/Uu interface of a cellular vehicle networking edge computing system based on Q learning. Through the Uu interface of the on-board C-V2X terminal and the PC-5 interface, tasks can be offloaded to the edge computing unit or a nearby vehicle. The team deduced the probability of successful transmission of vehicle-to-vehicle communication and constructed a constrained Markov decision process (CMDP) with the goal of minimizing system delay. Experimental results show that the proposed method is superior to traditional strategies in terms of time delay and energy consumption. Although the characteristics of the C-V2X link are considered, the complexity of the model is high, which may lead to the problem of large consumption of computing resources in practical applications.

Zhang *et al.*¹⁵ proposed a task offloading decision algorithm based on double deep Q-network (DDQN) and a prioritization-based resource allocation scheme. Through simulation experiments, the proposed algorithm is significantly optimized in terms of delay and energy consumption compared with traditional offloading algorithms. The algorithm design is complex and requires high computing resources, and may face the challenges of the network environment and vehicle dynamic changes in actual deployment. Gao *et al.*¹⁶ designed a partition-based hierarchical SDN vehicle-networking network architecture and proposed the ESLB algorithm, which is implemented based on deep Q network (DQN) and can balance the load of servers well while reducing the task completion delay. The scalability and robustness of the algorithm in practical applications need to be further verified, especially in large-scale vehicle networking environments. Liu *et al.*¹⁷ proposed a multi-objective optimization task offloading method, which comprehensively considered the service delay, load balancing and privacy information entropy of ES. Through the improved intensity Pareto evolution algorithm (SPEA2), the global optimal solution is obtained, which effectively improves the security and privacy protection level of task offloading. The algorithm may face the challenge of high computational complexity and high real-time requirements in the implementation process, so it needs to be balanced and optimized in practical applications. Zhang *et al.*¹⁸ proposed a “car-edge-cloud” task offloading architecture and designed a joint computational offloading method based on deep reinforcement learning. This method takes the energy consumption and delay of offloading as the entry point, and proposes an improved algorithm, which shows good convergence and performance optimization in simulation

experiments. Although the algorithm performs well in terms of latency and energy consumption, it may face challenges such as network latency and bandwidth limitations in practical applications, requiring further research and optimization.

Scholars have made remarkable progress in the deep learning-based method of offloading tasks on the edge computing of vehicle networking, but there are still some limitations in their respective studies. Future research needs to conduct in-depth exploration and optimization in the aspects of algorithm complexity, real-time, scalability, security and privacy protection, so as to promote the rapid development and wide application of vehicle networking technology.

3. Vehicle Networking System Model and Problem Description

3.1. Edge computing system of vehicle networking

The Internet of Vehicles edge computing system built in this study is carefully designed around the requirements of efficient and intelligent task processing in the Internet of Vehicles scenario. It is mainly composed of four key parts, namely, the vehicle unit layer, the edge computing layer, the intelligent unloading decision-making module and the cloud computing center. All parts work closely together to achieve efficient task processing and resource optimization in the Internet of Vehicles environment.^{19,20}

The onboard unit (OBU) layer serves as the data source and task initiator of the system, consisting of numerous OBUs. These OBUs are like the “nerve endings” of the vehicle, fully equipped with advanced sensors, communication modules and computing devices.

The edge computing layer is the key support of the system, which is composed of ES deployed at roadside units (RSUs) or specific geographical locations. These ES are like “smart stations” distributed along the road, with powerful computing, storage and network communication capabilities. When the edge server receives a task request from the vehicle-mounted unit, it will quickly analyze and evaluate the task. For tasks with high computational complexity or requiring the integration of multiple sources of data, ES performs partial computational tasks.

In order to achieve scientific and intelligent task offloading decision-making, the system introduces an intelligent offloading decision-making module based on deep learning and integrates it into the edge server. This module is like the “smart brain” of the system, using deep learning algorithms to perform real-time and accurate analysis of dynamic changes in the connected car environment. The system also includes a cloud computing center as a backup and advanced analysis support platform for the entire system.

3.2. Structure of edge computing server

3.2.1. Offloading unit

Task offloading in the field of vehicle networking refers to the powerful processing power of edge computing to make up for the computing resource limitations of

on-board terminals. Specifically, on-board terminals transfer some or all of their computing tasks to ES in order to take advantage of their higher computing efficiency. Once the edge server completes these tasks, the onboard terminal receives the processing results. This offloading mechanism is essentially a distributed computing model, the core of which is to reduce the energy consumption of on-board terminals by migrating computing tasks to ES. In this process, the onboard terminal sends data over the network to the edge server. Therefore, task offloading needs to consider the advantages of remote processing and the cost of data transmission. Task offloading is selected only when the local execution time of the task at the on-board terminal exceeds its total offloading time to the edge server (including communication delay and server processing time)²¹:

$$T_{\text{local}} > T_{\text{offloading}} = T_{\text{transfer}} + T_{\text{remote}}. \quad (1)$$

In the formula, T_{local} indicates local execution time, T_{remote} remote execution time, and T_{transfer} communication time. The energy gain is as follows:

$$E_{\text{local}} > E_{\text{remote}}, \quad (2)$$

where E_{local} is local execution energy consumption, E_{remote} is remote execution energy consumption in offloading. That is, offloading computing tasks to a remote server consumes less energy than computing tasks executed locally. Offloading is therefore beneficial for the execution of computing tasks.

In the decision-making process, the offloading unit (Fig. 1) not only needs to consider the offloading efficiency of a single task, but also needs to optimize the overall system resource utilization and task execution efficiency from a global perspective. The offload unit uses advanced deep learning algorithms, such as reinforcement learning or deep neural networks, to dynamically adjust the offload

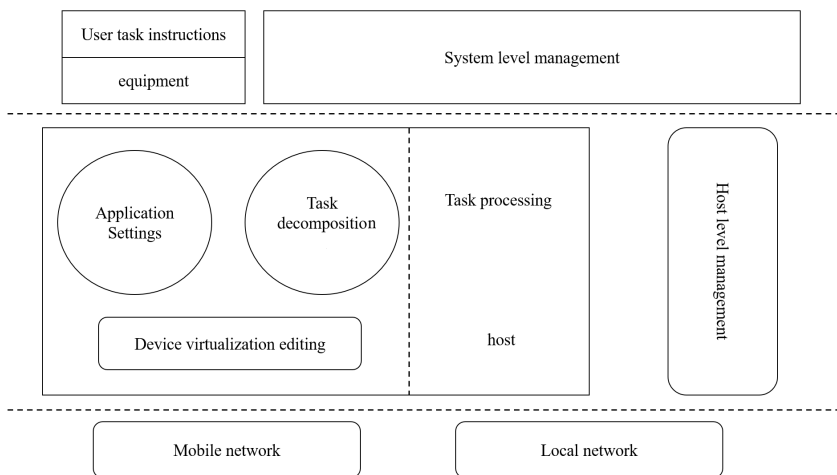


Fig. 1. Framework of uploading the unit frame.

strategy to the complex and variable task requirements and resource distribution in the connected vehicle environment.

Once an offload decision is made, the offload unit transmits the task data over the network to the selected edge computing node and monitors the execution of the task. During execution, the offload unit can also adjust resource allocation based on real-time feedback to ensure that tasks are completed on time and quality of service (QoS) requirements are met. At the same time, the offloading unit will also collect data during the execution of the task for the training and optimization of the deep learning model, so as to continuously improve the intelligence and accuracy of the offloading decision.

3.2.2. Dispatching unit

As the core component of the task-unloading method of Internet of Vehicles edge computing, the scheduling unit in this study plays a crucial role. Its main responsibility is to receive task requests from various OBUs, which cover various types such as data processing, information analysis and path planning. In order to ensure that urgent and high-priority tasks can be processed in a timely manner, the scheduling unit first classifies and prioritizes these tasks.^{22,23} This process has a direct and profound impact on the overall performance and user experience of the system.

On this basis, the scheduling unit also needs to intelligently offload tasks to suitable ES based on the characteristics of the tasks, the resource status of the ES, and the network conditions. This not only requires the scheduling unit to have comprehensive real-time information acquisition capabilities, but also to be able to quickly analyze and make decisions on complex situations. By introducing deep learning algorithms, the scheduling unit can intelligently predict the load situation of each edge server in the future, making the optimal task offloading decision and achieving reasonable allocation of tasks between the vehicle-mounted unit and the ES.

The scheduling unit is also responsible for monitoring the operational status of the entire system, promptly identifying and resolving potential bottlenecks and issues. For example, when an edge server experiences overload, the scheduling unit can quickly adjust the task offloading strategy and dynamically transfer some tasks to other idle or lightly loaded servers to ensure the stability and reliability of the system. Through the intelligent decision-making and scheduling mechanism, the scheduling unit not only realizes the efficient unloading of tasks but also optimizes the operating efficiency of the whole system, providing a solid guarantee for the effective implementation of the task unloading method of the Internet of Vehicles edge computing.

4. Task Offloading Method Based on VAE-LSTM

4.1. Problem description

The task offloading problem study involves multiple dimensions. First, the diversity of tasks. Tasks in the Internet of vehicles may include path planning, image

processing, real-time communication and other types, and each task has different requirements for computing resources, delay and bandwidth.²⁴ The second is the limitation of resources. The computing resources and network bandwidth of the edge server are limited, while the requests of the on-board terminal are dynamic, so it is a challenge to meet as many task requirements as possible under the circumstances of resource limitation. A variational autoencoders-long short-term memory (VESP-LSTM) task offloading model is established. Third, the dynamic nature of the environment, the vehicle networking environment is complex and changeable, vehicle movement, network conditions and other factors will affect the decision and execution of task offloading.

This research focuses on the task offloading problem of multiple on-board terminals in the networking of vehicles, which is an optimization problem involving multi-stage decision making. Our goal is to maximize the weighted sum of task processing efficiency, taking into account the actual power of the on-board terminal and the limitations of the available transmission bandwidth. If the system can identify the maximum weighted solution efficiency of all on-board terminal computing tasks, the optimal task offloading strategy can be formulated to minimize the task operation delay. The offloading decision-making problem is described as follows:

$$R_{\text{all}} = \max \left((w_i) \sum_{k=1}^{K-1} (x_k R_o + (1 - x_i) R_i) \right), \quad (3)$$

$$\begin{aligned} C1 : x_i &\in \{0, 1\} \\ C2 : \sum_{i=1}^N t_i (x_i p_i + (1 - x_i) k_i f_i^3) &\leq \sum_{i=1}^N \text{EC}_{\text{constraint}} \\ C3 : \sum_{i \in N} B_i &\leq B \end{aligned} \quad (4)$$

Let x_i be the variable representing the task offload decision, where $x_i = 0$ indicates that the task is selected to execute locally on the on-board terminal, and $x_i = 1$ indicates that the task is migrated to the mobile edge computing (MEC) server for execution. Under this framework, we set two key constraints: C2 means that the energy consumption of the task must not exceed the preset energy consumption threshold, and C3 means that the network bandwidth usage must also be controlled within the specified bandwidth limit. By solving the optimization problem, the study can determine the best offloading decision variables, and this process is reflected in the optimization model described in this paper.

4.2. Method design

4.2.1. Task unload communication

In the task of unloading Internet of Vehicles edge computing based on deep learning, the selection of the communication protocol is directly related to the system performance. Due to the need for fast transmission of task data to ES and timely

feedback of offloading decision information to OBUs, low latency and high reliability have become the core requirements of communication protocols.²⁵ This study analyzes various protocols, such as dedicated communication protocols for vehicle networking (such as C-V2X) and traditional network communication protocols (such as TCP/IP), aiming to match the optimal communication scheme for task offloading scenarios.

The limited spectrum resources pose a key challenge for task offloading. Under the current communication mechanism, when the channel is occupied, other vehicles cannot use it. Only when the occupying vehicle releases the channel resources can the channel be reused, which greatly limits the smoothness of task offloading. To overcome this bottleneck, this study proposes to interact the vehicle wireless resource pool with the vehicle-to-infrastructure (V2I) interface (see Fig. 2), dynamically allocate spectrum resources, improve resource utilization efficiency, and ensure efficient operation of the task offloading process.

According to the formula, we can calculate the signal-to-interference-plus-noise ratio (SINR) of vehicle C_i during the time period t_i as follows:

$$\text{SINR}_v(t_i) = \frac{P_i d_l^{-\delta} h^2}{N_0}. \quad (5)$$

Let P_i be the transmission power when vehicle C_i sends task data upward, and N_0 be the parameter standard deviation of additive white Gaussian noise (AWGN) when the vehicle transmits a signal. $d_l^{-\delta}$ represents the energy consumption generated by the vehicle through the channel when communicating with the RSU, d_l is the distance between the speed sensor in the server and the vehicle, δ is the path discount factor and h is the channel attenuation parameter of the upload link. As the number of cars increases, the SINR of an individual vehicle decreases, meaning that the vehicle needs to provide more energy to offload computing tasks.

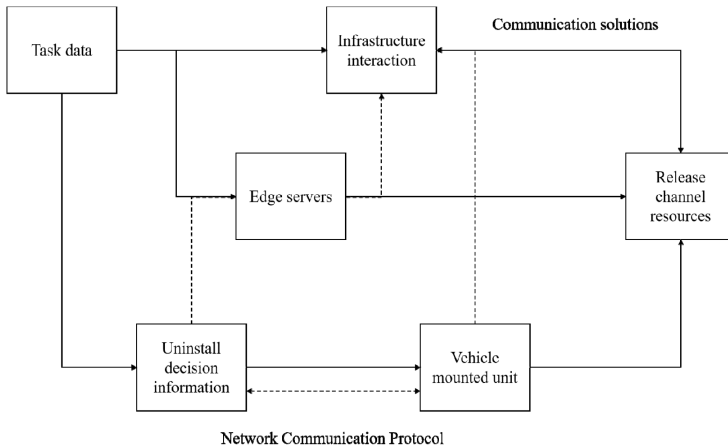


Fig. 2. The vehicle wireless resources and infrastructure interaction process.

The vehicle establishes a wireless link with the server through a direct connection to achieve communication. Due to the high speed of vehicle movement on the road, the impact on the server is relatively small. According to Shannon's formula, the study can calculate the task processing rate of the upload path. To simplify the calculation, we approximate the calculated offloading rate of a vehicle to its task processing rate, as follows:

$$R_{UP} = B_{UP} \log_2(1 + \text{SINR}_\nu(t_i)), \quad (6)$$

where B_{UP} indicates the bandwidth of the upload link channel. In this paper, it is assumed that the vehicles in the proposed vehicle-networking system are traveling at uniform speed in a straight line in the road, the i car is represented by C_i , and the vehicle speed is v_i . Over time, constant changes in the vehicle's position affect the distance between the vehicle and the server center:

$$d_t(t) = \sqrt{r^2 + \left(\frac{s}{2} - v_i t\right)^2}, \quad (7)$$

where r represents the distance between the vehicle and the central location of the server at time t , and s represents the radius of the server coverage area. The average transfer rate is used to replace the data transfer efficiency of computing tasks uploaded to the MEC server:

$$\overline{R_{UP}} = \frac{\int_0^{t_{\text{stay}}} R_{UP}(t) dt}{t_{\text{stay}}}, \quad (8)$$

$$t_{\text{stay}} = \frac{s - l}{v_i}, \quad (9)$$

where t_{stay} indicates the time, the vehicle is in the range of a server. l is the distance between the vehicle and the rear boundary of the RSU coverage area when the vehicle initiates an offload to the MEC server.

4.2.2. Edge task unload processing

The starting point of the edge task offloading process is when the OBUs detect computationally intensive or latency-sensitive tasks and their related data that need to be offloaded. These tasks and data are transmitted to nearby ES through efficient communication protocols. Once the edge server receives these tasks, it will immediately start the uninstallation processing module. The primary task of this module is to perform preliminary analysis and classification of tasks to determine their characteristics, priority, and required resources (as shown in Fig. 3).

The process of offloading tasks to edge servers (ES) for remote processing can be divided into three stages:

- (1) OBU uploads task data to RSU through wireless channels.
- (2) ES utilizes its idle virtual machine resources to perform these tasks.

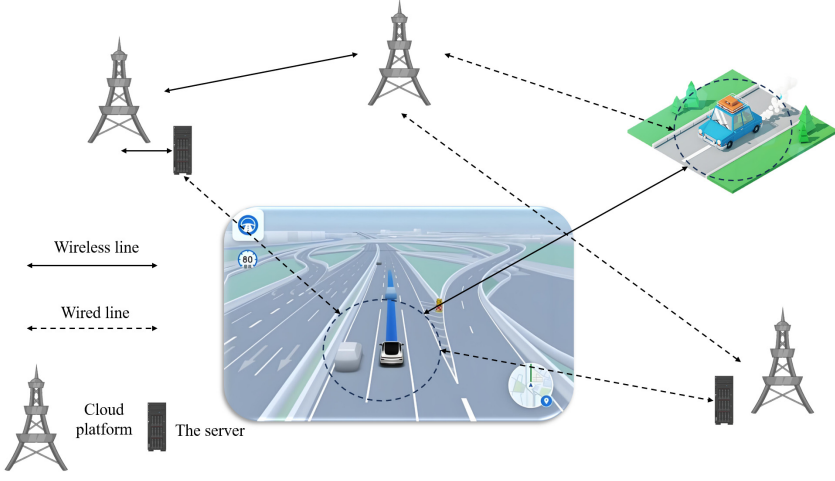


Fig. 3. Networked vehicle terminal task processing.

- (3) The RSU feeds back the calculation results to the OBU's through the downlink wireless channel.

Due to the fact that downlink transmission only involves feedback of calculation results, its data volume is relatively small. Therefore, in this study, we ignored the impact of step 3 on latency and energy consumption.

In step 1, the study assumes that users within R_m coverage share bandwidth resources via single-carrier orthogonal frequency division multiple access (SC-FDMA) technology. This study further assumes that the total bandwidth B is evenly distributed among the users who need to offload the task:

$$\beta = \frac{B}{\sum_{i \in n} x_i}, \quad (10)$$

$$R_{nm}(t) = \beta \cdot \log_2 \left(l + \frac{p_n^{\text{up}} \cdot H_{nm}(t)}{N_0} \right), \quad (11)$$

where p_n^{up} is the vehicle uplink transmitting power, $H_{nm}(t)$ is the wireless channel gain between U_n and RSU, and N_0 is the standard deviation of Gaussian white noise in the channel environment. U_n is in a non-static state within the R_m coverage, and the position of RSU is defined as (x_m, y_m) , so the time-varying distance between U_n and RSU can be obtained as:

$$l_{nm}(t) = \sqrt{(x_n(t) - x_m)^2 + (y_n(t) - y_m)^2}. \quad (12)$$

Once an offload decision is made, the edge server quickly executes the task or assigns it to the appropriate computing resources for processing. During the processing, the edge task offloading processing module also continuously monitors the execution

status and resource usage of the task to ensure that the task can be completed on time and meet the quality-of-service requirements. At the same time, the module will also collect data during task execution for the training and optimization of deep learning models, so as to continuously improve the intelligence and accuracy of offloading decisions.

4.3. Algorithm implementation

In this research on task unloading of Internet of Vehicles edge computing based on deep learning, this research focuses on resource management strategies in the critical scenario of computing resource surplus. The so-called computing resource surplus refers to the situation where the available computing resources in the connected vehicle environment exceed the actual required computing resource scale for current connected vehicle applications. This phenomenon is not uncommon in vehicle networking scenarios where vehicles are densely distributed.

In the context of connected vehicles, vehicles are in a high-speed driving state, which poses many uncertainties and potential risks to the task offloading process. When vehicles unload tasks during movement, they may encounter problems such as network signal fluctuations, communication interruptions and node failures, which can lead to task unloading failures.^{26,27} In order to more efficiently utilize excess computing resources and significantly improve the reliability and overall performance of task offloading in dynamic environments, an innovative multi-channel offloading mechanism has been designed. The core idea of this mechanism is to reduce the impact of single-node failures or communication issues on the success rate of task offloading by flexibly allocating and fully utilizing computing resources.

$$\max_{1 \leq \varepsilon \leq \bar{\varepsilon}} \xi = 1 - \varepsilon^\xi, \quad (13)$$

$$\text{s.t.} \begin{cases} \varepsilon \in [0, 1], \\ \xi = \sum_{1 \leq j \leq K_i} C_j^{\text{comp}} \sum_{1 \leq i \leq K_i} C_i^{\text{comp}}. \end{cases} \quad (14)$$

The anti-risk capability ξ can fully reflect the ability to successfully unload computing tasks in the dynamic change scenario of vehicle networking resources. Risk factor ε and multiplexing factor are introduced.

Study proposes a value-first offloading mechanism to guide offloading decisions for computing tasks in networked vehicles. Specifically, for the diverse business needs in the Internet of Vehicles, study classify computing tasks into different value levels based on their urgency and QoS requirements. Connected vehicle applications that involve vehicle safety are typically assigned a higher value rating, while connected vehicle applications that focus on entertainment are typically assigned a lower value rating. Based on this value hierarchy, study reconstruct the objective function to

achieve a better task offloading strategy. This objective function is defined as follows:

$$\max \sum_{1 \leq \tilde{i} \leq \tilde{K}_1} R_{\tilde{i}}, \quad (15)$$

$$\text{s.t.} \begin{cases} C_1 - C_6 \text{ in } (4 - 1), \\ C_7 : V_i \geq V_{i+1}, \quad (1 \leq \tilde{i} \leq \tilde{K}_i), \\ C_8 : 0 \leq \tilde{K}_t \leq K_t, \\ C_9 : \sum_{1 \leq i \leq \tilde{K}_t} C_t^{\text{comp}} \leq C_{\text{sum}}^{\text{comp}}, \end{cases} \quad (16)$$

where $\{V_1, \dots, V_{i+1}, \dots, V_{Kt}\}$ is the set of requirements that will eventually be met for the computing task offload, $\tilde{i}$ and $\tilde{K}_i$ represents the maximum number of vehicles per task and that can be supported by the task. In terms of constraints, C_7 represents that high-value mission vehicle requests will be met preferentially, C_8 defines the sufficient range, and C_9 shows that the allocated computing resources cannot exceed $C_{\text{sum}}^{\text{comp}}$, the maximum value of the overall available resources in the model.

The workflow of the multi-path offloading mechanism is as follows: First, the system will accurately allocate the required computing resources to the task vehicles based on their real-time needs. This step is based on a comprehensive analysis of multidimensional factors such as vehicle task types, data volume and processing latency requirements, ensuring that each task vehicle can obtain resources that meet its basic computing needs. For example, for autonomous vehicle tasks that require real-time image recognition, the system will allocate sufficient computing power and storage resources based on the size of the image data and the complexity of the recognition algorithm.^{28,29}

After completing the basic resource allocation, the multi-path offloading mechanism will initiate a secondary allocation process for the remaining computing resources that have not been fully utilized in the system. As long as one of these task nodes can successfully complete the computing task, the sender can obtain the required computing results in a timely manner.

From the perspective of resource utilization, the multi-channel offloading mechanism achieves deep optimization of excess computing resources. It avoids waste caused by idle resources, allowing every computing resource in the system to be fully utilized and improving resource utilization.

4.4. Data collection, training and optimization

4.4.1. Data set

Hyper-Erlang distribution is used to simulate vehicle residence time in an edge computing environment.³⁰ This distribution allows us to discretize the continuous time into a base time slice, where the execution time, transmission time and queuing time are all integral multiples of the base time slice. This dataset helped us evaluate the performance of different offloading strategies for different vehicle residence times.

To simulate diverse connected vehicle services, we classify computing tasks into different value levels based on their urgency and service quality requirements. Security applications are assigned a high value rating, while entertainment applications are assigned a low value rating. This dataset allows us to assess the effectiveness of the value-first offloading mechanism.

The study considers a vehicle-connected scenario of dynamic traffic flow and dynamic networks, where MEC and cloud computing are used to collaboratively handle delay-sensitive and compute-intensive tasks in a vehicle-connected. The study uses a dataset that includes task processing latency and transfer rate constraints to minimize the system cost of processing tasks.

4.4.2. Data training and optimization

In terms of data training, the study has collected a large amount of historical data in the connected vehicle environment, including key indicators, such as the computation amount of tasks, delay requirements, computing power of ES, network bandwidth and transmission delay. After the data is preprocessed and cleaned, it is used to train deep learning models. The study uses a multi-layer neural network structure and constantly adjusts the weights and bias parameters of the model to minimize prediction errors through optimization techniques such as backpropagation algorithms and gradient descent. At the same time, in order to avoid the overfitting problem, the study also introduces the regularization term and dropout mechanism to enhance the generalization ability of the model. The experimental parameter settings are shown in Table 1.

In terms of model optimization, the study has adopted a variety of strategies to further improve the performance of the model. First, the study adopted a transfer learning strategy, using the model already trained in the relevant field as a pre-training model, and fine-tuning it to adapt to the connected vehicle environment. Second, the study introduces a reinforcement learning mechanism to simulate the task offloading process in the networked vehicle environment, so that the model can learn and optimize the offloading strategy through continuous trial and error. This approach allows the model to make more intelligent decisions based on changes in the

Table 1. Experimental parameter settings.

Parameter items	Set value or description
DQN network structure	A three-layer fully connected network with a single hidden layer
Number of hidden layer nodes	50
Hidden activation function	ReLu
Experience pool size	2000
Action selection strategy	E-greedy strategy, with an ε value of 0.9, gradually increasing from 0 in the early stage, with an increment of 0.0005
Batch-size	64
Learning rate	0.0001
Discount factor	0.9

actual environment. In addition, the study also adopts distributed training technology, which uses multiple machines to compute in parallel to accelerate the training process of the model and improve the training efficiency.

5. Simulation and Analysis

5.1. Task offloading success rate

By comparing the success rates of different deep learning algorithms in task offloading (see Fig. 4 below), the performance of each algorithm can be intuitively evaluated. This helps researchers understand which algorithms perform better in certain scenarios, allowing them to make more informed choices in real-world applications. By comparing the task offloading success rates of different deep learning (see Fig. 5 below), the study can find the advantages and disadvantages of different algorithms under different conditions. This helps researchers optimize offloading strategies according to specific scenarios and requirements, and improve the efficiency and performance of the overall system. Some deep learning algorithms have a higher task offloading success rate in certain scenarios, which depends on the characteristics of the algorithm, model structure and training data.

Through comparative analysis, it is found that the VAE-LSTM method is always superior to the CNN, RNN, FNN and GAN four methods in the whole network delay interval, which highlights the excellent accuracy and robustness in the Internet of Vehicles edge computing task unloading scenario. On the other hand, the performance of GAN methods is poor, especially in high network latency environments, where the success rate of task offloading decreases significantly. This may be attributed to the limitations of GAN models' generalization ability when dealing with complex tasks. The performance of CNN, RNN and FNN methods is between VAE-LSTM and GAN, showing relatively stable performance under different network

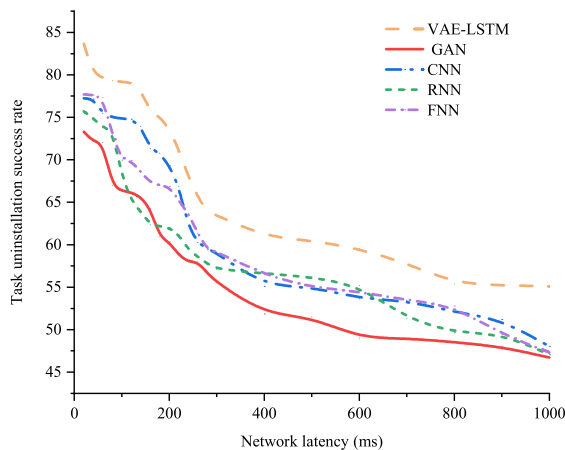


Fig. 4. Task offloading success rate.

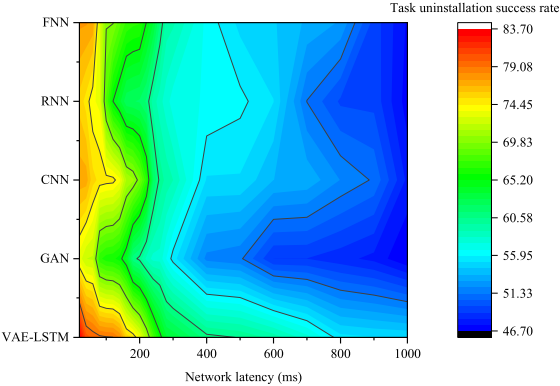


Fig. 5. Offloading success rate contour color filling.

delay conditions, but overall performance is still inferior to VAE-LSTM. When the network latency is low, the VAE-LSTM method maintains a stable task offloading success rate of over 80%, showing significant advantages. Although other methods can achieve a certain success rate, they are all lower than VAE-LSTM. The performance of the GAN method is relatively stable under these conditions, but there is still a gap compared to other methods. As network latency increases, the success rates of all methods decrease, while VAE-LSTM has a smaller decrease and continues to maintain a high success rate. The performance differences between CNN, RNN and FNN methods are gradually narrowing, but overall, it is still difficult to surpass VAE-LSTM methods.

5.2. Quality of service

By comparing the QoS of different deep learning methods in IoT edge computing task offloading (Fig. 6), the effectiveness of each method can be intuitively evaluated. QoS is an important index to measure system performance, including delay,

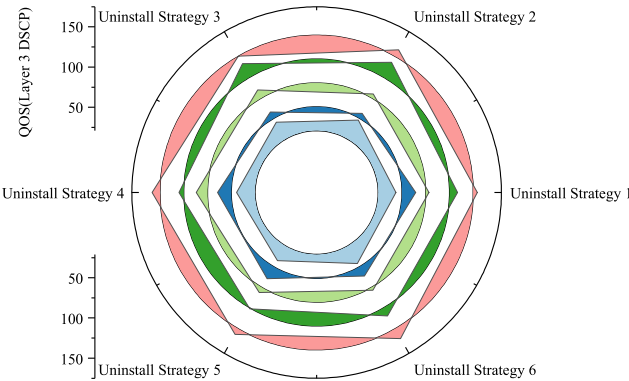


Fig. 6. QoS results.

throughput, reliability, etc. It is directly related to user experience and system efficiency. Therefore, comparing QoS can be helpful in understanding which deep learning methods provide better performance in real-world applications. Different deep learning methods may be suitable for different scenarios and task requirements. By comparing QoS, you can find the advantages and disadvantages of different methods under different conditions, and optimize the task uninstillation strategy. For example, in some cases, certain methods may be able to significantly reduce latency, while in other cases, other methods may be more effective at improving throughput. Through a comparative study, it can provide a scientific basis for task offloading strategies in different scenarios.

It can be seen from the data that different uninstillation strategies have a significant impact on the QoS scores of the five methods. This shows that the choice of offloading strategy is very important to improve the service quality of the offloading of the edge computing task of the Internet of vehicles. The GAN method has a relatively low QoS score, especially under some unload policies (such as unload policies 2 and 3), and its score is much lower than that of the VESP-LSTM method. This may be related to the lack of stability and robustness of GAN models when dealing with complex tasks. The CNN, RNN and FNN methods have QoS scores between VESPA-LSTM and GAN. Their performance is relatively stable under different offloading strategies, but the overall performance is slightly lower than that of the VESP-LSTM method. Under uninstillation policy 1 and uninstillation policy 4, the QoS score of VAE-LSTM is relatively high, and the gap is obvious with other methods. This indicates that the VESP-LSTM method has high adaptability and stability when dealing with similar offloading strategies. The QoS score of the GAN method is also relatively high under these two strategies, but it is still lower than that of the VEVA-LSTM method. This indicates that the GAN method may have some applicability under some specific strategies, but the overall performance still needs to be improved. Under all offloading strategies, the VESP-LSTM method has a higher QoS score than the other four methods. This indicates that the VEVE-LSTM method has a high service quality in handling the offloading of IoT edge computing tasks.

5.3. Task offloading ratio

How to determine which tasks to unload and how many tasks to unload, that is, the task unload ratio, is a complex and critical problem. As a powerful tool, deep learning is widely used in decision making and optimization of task offloading. By comparing the performance of different deep learning models on the task offload ratio (Fig. 7 below), the performance of each model can be visually assessed. This helps to understand which models are more accurate in determining task offload ratios in specific scenarios, allowing for more informed choices in real-world applications. By comparing the task offloading ratio of different deep learning models, the study can find the advantages and disadvantages of each model under different conditions, so

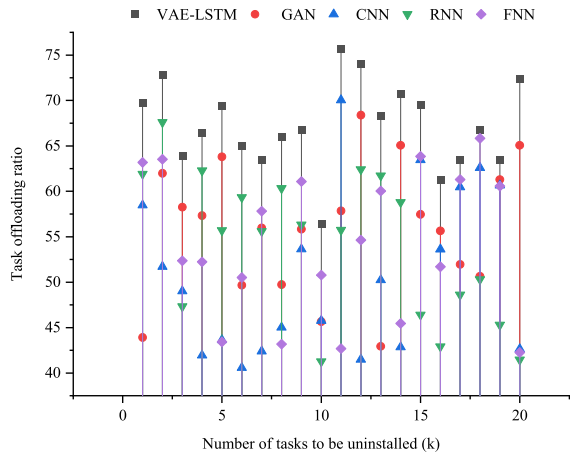


Fig. 7. Task offloading ratio results.

as to optimize the offloading strategy. For example, in some scenarios, certain models may be able to more accurately predict which tasks should be offloaded to improve the overall efficiency of the system.

Among all the tasks to be unloaded, the VEEVE-LSTM method has a higher offloading ratio than the other four methods. This indicates that the VEEVE-LSTM method has higher efficiency and accuracy in handling the offloading of IoT edge computing tasks. The task offloading ratio of the GAN method fluctuates greatly under different tasks to be unloaded, and the overall performance is relatively poor. This may be related to the lack of stability and robustness of GAN models when dealing with complex tasks. The task offloading ratio of CNN, RNN and FNN methods is between VAE-LSTM and GAN, and the performance is relatively stable under different task loads. Overall, however, their performance is still slightly lower than that of the VEEVE-LSTM method. Under the condition of low task load, the task offloading ratio of the VEEVE-LSTM method is generally higher, and the gap between the VEEVE-LSTM method and other methods is obvious. This shows that the VEEVE-LSTM method has high efficiency and accuracy when dealing with a small number of tasks. Under this condition, the task offloading ratio of the GAN method is relatively low and fluctuates greatly. This may be related to the inadequacy of GAN models in handling a small number of tasks.

Under medium task load conditions, the VEEVE-LSTM method still has a higher task offloading ratio than other methods, but the gap is reduced. This indicates that the VAE-LSTM method still has some advantages in dealing with a medium number of tasks, but other methods may also show some competitiveness. The task offload ratio of CNN, RNN and FNN methods is relatively stable under this condition, but still lower than that of VEEVE-LSTM methods overall. The performance of the GAN method is also improved under this condition, but it still needs further optimization.

Under the condition of high task load, the task offloading ratio of the VESPA-LSTM method is still at a high level, and the gap between the VESPA-LSTM method and other methods is further widening. This shows that the VESPA-LSTM method has significant advantages when dealing with a large number of tasks. The performance of the GAN method is improved under the condition of high task load, but it is still far lower than that of the VAE-LSTM method. The performance of CNN, RNN and FNN methods is also relatively stable under high task load conditions, but the overall performance still needs to be improved.

5.4. MAE results

MAE calculates the average of the absolute value of the difference between the predicted value and the true value, providing an intuitive measure of the model's prediction bias. In IoT edge computing task offloading, MAE can measure the accuracy of the model in predicting which tasks should be offloaded and which tasks should be handled locally. The smaller MAE value means that the prediction of the model is more accurate and can better adapt to the dynamic change of the vehicle networking environment. By comparing the MAE of different deep learning models (see Figs. 8 and 9 below), it is possible to discover which models have higher prediction accuracy in a given scenario. This helps optimize the offloading strategy and improve the overall efficiency of the system. In some scenarios, certain models may be able to more accurately predict which tasks offloading to ES will result in greater performance gains, and these models may be selected for offloading decisions in these scenarios.

As the number of tasks to be unloaded increases, the MAE of the five methods does not show an obvious linear relationship. This indicates that MAE is not only affected by the number of tasks to be unloaded, but also by many other factors, such

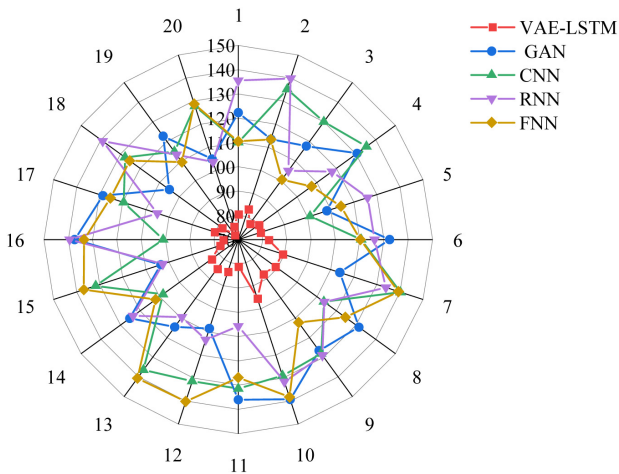


Fig. 8. Spider web form of task offloading ratio.

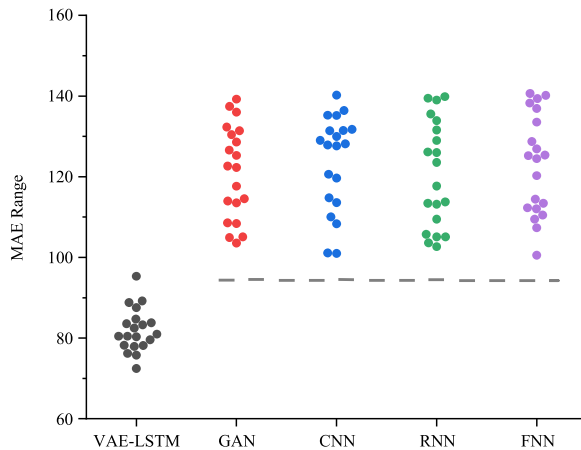


Fig. 9. Mean absolute error (MAE) experiment results.

as model complexity, computing resources and task types. The MAE of the GAN method fluctuates greatly under different tasks to be unloaded, and the overall performance is relatively poor. This may be related to the lack of stability and robustness of GAN models when dealing with complex tasks. The MAE of CNN, RNN and FNN methods is between VAE-LSTM and GAN, and the performance is relatively stable under different offloading tasks. Overall, however, their performance is still slightly lower than that of the VESP-LSTM method. The MAE of the VAE-LSTM method is generally lower than that of the other four methods under all offloading tasks. This indicates that the VEVE-LSTM method has higher prediction accuracy and stability in handling the offloading of IoT edge computing tasks.

6. Results and Discussion

In this paper, the study evaluates the performance of five different deep learning models, including VAE-LSTM, GAN, CNN, RNN and FNN, in the offloading of edge computing tasks in the Internet of Vehicles.

The experimental results show that the VESP-LSTM model exhibits lower MAE under all the offloading tasks, which indicates that it has high prediction accuracy and stability. Compared with other models, VEVE-LSTM can better adapt to the dynamic network environment when handling the offloading of IoT edge computing tasks, so as to provide more accurate offloading decisions. This advantage is particularly evident when the task volume is large, further demonstrating the robustness of VESPA-LSTM in handling complex tasks. The experimental performance of the GAN model is relatively poor, and its MAE value fluctuates greatly under different tasks, which may be related to the stability and adaptability of GAN in dealing with complex tasks. The performance of CNN, RNN and FNN models is between VEVE-LSTM and GAN, and although they also have some predictive ability, their overall performance and stability are still slightly worse than VEVE-LSTM.

This paper proposes a task-unloading method for edge computing of the Internet of Vehicles based on deep learning, and verifies its effectiveness in reducing task delay and energy consumption through experiments. The research results show that the deep learning model can dynamically adjust task unloading decisions according to real-time network status and task characteristics, thus significantly improving the performance of the Internet of Vehicles system. However, the adaptability of the model under complex network topology still has some limitations and needs further optimization and improvement. Future research will focus on the following directions: optimizing the structure and parameters of the deep learning model, improving its robustness and adaptability in complex environments. Explore multi-objective optimization strategies, taking into account system security, privacy protection and resource utilization. Combined with blockchain and other emerging technologies, build a more intelligent and secure edge computing system of the Internet of Vehicles. These research directions will provide a more comprehensive and effective solution for the task unloading of Internet of Vehicles edge computing.

Acknowledgment

This work was supported by the Key R&D and Promotion Special Program of Henan Province (Grant No. 222102110392), by the Key Scientific Research Project of Colleges and Universities in Henan Province (Grant No. 23A460024), and by the Key Scientific Research Project of Colleges and Universities in Henan Province (Grant No. 24B460030), and by the Key Scientific Research Project of Colleges and Universities in Henan Province (Grant No. 24A460023).

ORCID

Guangbo Xiang  <https://orcid.org/0009-0001-9635-4908>

References

1. K. Yu *et al.*, Hybrid quantum classical optimization for low-carbon sustainable edge architecture in RIS-assisted AIoT healthcare systems, *IEEE Internet Things J.* **11** (2024) 38987–38998.
2. J. Yang *et al.*, Cloud-edge-end collaborative caching based on graph learning for cyber-physical virtual reality, *IEEE Syst. J.* **17** (2023) 5097–5108, doi: 10.1109/JSYST.2023.3262255.
3. Z. Lin, H. Cui and Y. Liu, Distributed deep learning based on edge computing over internet of vehicles: Overview, applications, challenges, *IEEE Access* **12** (2024) 133734–133747.
4. K. Wang, X. Wang and X. Liu, A high reliable computing offloading strategy using deep reinforcement learning for IOVS in edge computing, *J. Grid Comput.* **19** (2021) 15.
5. M. Yan *et al.*, Edge computing task offloading optimization for a UAV-assisted Internet of Vehicles via deep reinforcement learning, *IEEE Trans. Veh. Technol.* **73** (2023) 5647–5658.
6. Y. Wang *et al.*, A new edge computing method based on deep learning in intelligent transportation system, *IEEE Trans. Veh. Technol.* **74** (2024) 2210–2219.

7. J. Wang and L. Wang, Mobile edge computing task distribution and offloading algorithm based on deep reinforcement learning in internet of vehicles, *J. Ambient Intell. Humaniz. Comput.* **12** (2021) 1–11.
8. S. Cao et al., Reinforcement learning based tasks offloading in vehicular edge computing networks, *Comput. Netw.* **234** (2023) 109894.
9. D. Zhang et al., Task offloading method of edge computing in internet of vehicles based on deep reinforcement learning, *Clust. Comput.* **25** (2022) 1175–1187.
10. L. Li, B. Zhi and S. Li, Data traffic offloading method of Internet of Things based on mobile edge computing, *Meas.: Sensors* **34** (2024) 101253.
11. W. Yang et al., Deep reinforcement learning-based low-latency task offloading for mobile-edge computing networks, *Appl. Soft Comput.* **166** (2024) 112164.
12. L. Yan et al., A task offloading algorithm with cloud edge jointly load balance optimization based on deep reinforcement learning for unmanned surface vehicles, *IEEE Access* **10** (2022) 16566–16576.
13. Y. Zhou et al., An indoor blind area-oriented autonomous robotic path planning approach using deep reinforcement learning, *Expert Syst. Appl.* **254** (2024) 124277.
14. Y. Liu et al., Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks, *IEEE Trans. Veh. Technol.* **68** (2019) 11158–11168.
15. S. Zhang et al., Dynamic deployment method based on double deep Q-network in UAV-assisted MEC systems, *J. Cloud Comput.* **12** (2023) 130.
16. Y. Gao et al., A hierarchical routing scheme with load balancing in software defined vehicular ad hoc networks, *IEEE Access* **6** (2018) 73774–73785.
17. J. Liu et al., Distributed location-aware task offloading in multi-UAVs enabled edge computing, *IEEE Access* **10** (2022) 72416–72428.
18. X. Zhang, X. Zhang and W. Yang, Joint offloading and resource allocation using deep reinforcement learning in mobile edge computing, *IEEE Trans. Netw. Sci. Eng.* **9** (2022) 3454–3466.
19. Z. Guo et al., Industrial 6G-IoT and machine learning-supported intelligent sensing framework for indicator control strategy in sewage treatment process, *IEEE Internet Things J.* **11** (2024) 29308–29320.
20. Z. Wang, T. Lv and Z. Chang, Computation offloading and resource allocation based on distributed deep learning and software defined mobile edge computing, *Comput. Netw.* **205** (2022) 108732.
21. J. Liu et al., A task oriented computation offloading algorithm for intelligent vehicle network with mobile edge computing, *IEEE Access* **7** (2019) 180491–180502.
22. Y. Yuan, M. A. Ikbal and A. Alam, Task offloading algorithm for mobile edge computing based on artificial intelligence, *Electrica* **22** (2022) 387–394.
23. X. Zhou et al., Edge computation offloading with content caching in 6G-enabled IOV, *IEEE Trans. Intell. Transp. Syst.* **25** (2023) 2733–2747.
24. J. Wu et al., Resource allocation for delay-sensitive vehicle-to-multi-edges (V2Es) communications in vehicular networks: A multi-agent deep reinforcement learning approach, *IEEE Trans. Netw. Sci. Eng.* **8** (2021) 1873–1886.
25. M. J. Peixoto and A. Azim, Design and development of a machine learning-based task orchestrator for intelligent systems on edge networks, *IEEE Access* **11** (2023) 33049–33060.
26. H. Qi, X. Mu and Y. Shi, A task offloading strategy of IoT devices using deep reinforcement learning based on mobile cloud computing environment, *Wirel. Netw.* **30** (2024) 3587–3597.
27. S. Zhou et al., Short-term traffic flow prediction of the smart city using 5G internet of vehicles based on edge computing, *IEEE Trans. Intell. Transp. Syst.* **24** (2022) 2229–2238.

28. L. Zhao *et al.*, A digital twin-assisted intelligent partial offloading approach for vehicular edge computing, *IEEE J. Sel. Areas Commun.* **41** (2023) 3386–3400.
29. J. Lv *et al.*, Mobile edge computing oriented multi-agent cooperative routing algorithm: A DRL-based approach, *Fractals* **31** (2023) 2340099.
30. Z. Wu *et al.*, Deep reinforcement learning-based task offloading and load balancing for vehicular edge computing, *Electronics* **13** (2024) 1511.